Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_09_)

Implementation of a Register File using Model Sim

Student Name: ___Areeb Fatima_____ Roll No: _____24I-6543_____ Section: _B_

Date performed: ___21st October_____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task, we basically wrote the code for 4x4 register file, firstly we wrote the module for decoder, after that we wrote the code for mux 4x1 and then we made the separate module for 4 bit register by putting decoder output as input to register file and data input also after that we made the module for 4x4 register file and we call decoder 2to4 for giving output to register input then call 4 register files and after that we read the result by calling the two mux modules.

Code:

```
module decoder_2_to4(wr, write_enable, dec_out);

input [1:0] wr;

input write_enable;

output reg [3:0] dec_out;

   always @(*) begin

      if(write_enable) begin

         case(wr)

            2'b00: dec_out= 4'b0001;

            2'b01: dec_out= 4'b0010;

            2'b10: dec_out= 4'b0100;

            2'b11: dec_out= 4'b1000;

            default: dec_out= 4'b0000;

          endcase

      end

      else begin

         dec_out= 4'b0000;

      end

   end
```

```
endmodule

module mux4_1(in0, in1, in2, in3, sel, out);

input [3:0] in0, in1, in2, in3;

input [1:0] sel;

output reg [3:0] out;

   always @(*) begin

   if (sel==2'b00)

      out= in0;

   else if (sel==2'b01)

      out= in1;

   else if (sel==2'b10)

      out= in2;

   else if (sel==2'b11)

      out= in3;

   else

      out= 4'b0000;

   end

endmodule

module register_4bit(load, clk, d, q);

input clk, load; //decoder output (load)

input [3:0] d; //data input

output reg [3:0] q;

   initial begin
```

```
    q=4'b0000;
  end
  always @ (posedge clk) begin
    if(load)
      q <= d;
  end
endmodule
module register_file_4x4(data_in, wr, write_enable, rd1, rd2, clk, data_out1, data_out2);
input write_enable, clk;
input [1:0] wr, rd1, rd2;
input [3:0] data_in;
output [3:0] data_out1, data_out2;
wire [3:0] dec_out, q0, q1, q2, q3;
//instantiate decoder
decoder_2_to4 decoder(wr, write_enable, dec_out);
//instantiate four 4-bit registers
register_4bit reg0(.load(dec_out[0]), .clk(clk), .d(data_in), .q(q0));
register_4bit reg1(.load(dec_out[1]), .clk(clk), .d(data_in), .q(q1));
register_4bit reg2(.load(dec_out[2]), .clk(clk), .d(data_in), .q(q2));
register_4bit reg3(.load(dec_out[3]), .clk(clk), .d(data_in), .q(q3));
//instantiate two 4-to-1 multiplexers
mux4_1 mux1(q0, q1, q2, q3, rd1, data_out1);
mux4_1 mux2(q0, q1, q2, q3, rd2, data_out2);
endmodule
```

Output:

```
# End time: 03:33:55 on Oct 21,2026, Elapsed time: 0:06:48
# Errors: 0, Warnings: 0
# Compile of reg_4x1_task01.v was successful.

ModelSim>
```

Task 2:

Description:

In this task, we basically simulate the register file by writing 6 in register 1 and 3 in register 2 and 13 to register 3 by enabling write to 1 and read the register 1 and 2 on dataout1 and dataout2 by enabling write to 0 mode and read by calling rd1 and rd2 for 1 and 2 select lines and then the data out will be respectively the input we wrote at 1 as 6 and 2 as 3 and 13 in register 3.

Code:

```
module testbench_reg_file();

reg write_enable, clk;

reg [1:0] wr, rd1, rd2;

reg [3:0] data_in;

wire [3:0] data_out1, data_out2;

register_file_4x4 uut(data_in, wr, write_enable, rd1, rd2, clk, data_out1, data_out2);

always

#5 clk= ~clk;

initial

begin

data_in=0;

wr=0;

write_enable=0;

rd1=0;
```

```
rd2=0;

clk=0;

#10

write_enable=1;

wr= 2'b01;

data_in= 4'd6;

#10

wr= 2'b10;

data_in= 4'd3;

#10

wr= 2'b11;

data_in= 4'd13;

#10

write_enable=0;

rd1= 2'b01;

rd2= 2'b10;

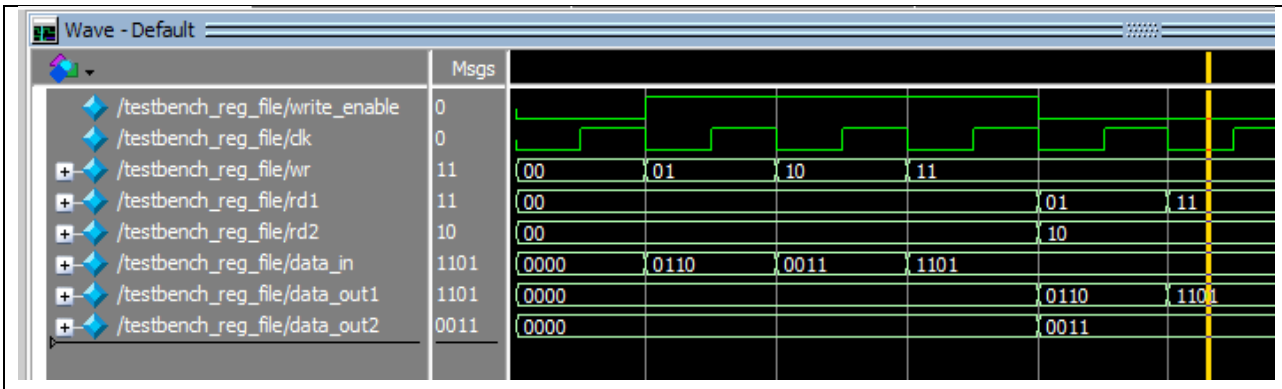
#10

rd1= 2'b11;

end

endmodule
```

Output:



Task 3:

Description:

In this task, we must design the 32-bit register file, firstly I tried to implement this by using the structural modeling but then I realize it to be made as the 32-bit register by running the loop as if reset then return 0 else if write enable then wrote reg_file select line as data_in and it works as decoder and then assig data_out to the ref_file rd1 selection operation which works as mux and by giving the 32-bit range we made it and wrote testbench simultaneously by enabling clock at posedge.

Code:

```
//module decoder_5_to32(wr, write_enable, dec_out);

//input [4:0] wr;

//input write_enable;

//output reg [31:0] dec_out;

//    always @(*) begin

//        dec_out= 32'h00000000;

//      if(write_enable &&(wr!= 5'd0))

//          dec_out[wr]= 1'b1;

//    end

//endmodule

//module mux32_1(sel, out);

//reg [31:0] reg_data [0:31]; //32-bit data n 32 registers
```

```
//input [4:0] sel;

//output reg [31:0] out;

//    always @(*) begin

//        if(sel==5'd0)

//            out= 32'h00000000;

//        else

//        out= reg_data[sel];

//    end

//endmodule

//module register_32bit(load, clk, reset, d, q);

//input clk, load, reset; //decoder output (load)

//input [31:0] d; //data input

//output reg [31:0] q;

//    always @ (posedge clk) begin

//        if(reset)

//            q <= 32'h00000000;

//        else if(load)

//            q <= d;

//    end

//endmodule

module register_file_32x32(clk, reset, write_enable, wr, rd1, rd2, data_in, data_out1, data_out2);

input write_enable, clk, reset;

input [4:0] wr, rd1, rd2;
```

```
input [31:0] data_in;

output [31:0] data_out1, data_out2;

reg [31:0] reg_file[31:0];

integer i;

   always @ (posedge clk) begin

      if(reset) begin

         for(i=0; i<32; i=i+1)

            reg_file[i] <= 32'b0;

      end

      else if(write_enable) begin

         if(wr != 5'd0)

            reg_file[wr] <= data_in;

      end

   end

//Asynchronous Read

assign data_out1 = (rd1 == 5'd0) ? 32'b0 : reg_file[rd1];

assign data_out2 = (rd2 == 5'd0) ? 32'b0 : reg_file[rd2];

endmodule

module testbench_32Bit_reg_file();

reg write_enable, clk, reset;

reg [4:0] wr, rd1, rd2;

reg [31:0] data_in;

wire [31:0] data_out1, data_out2;
```

```
register_file_32x32 uut(clk, reset, write_enable, wr, rd1, rd2, data_in, data_out1, data_out2);

always

#5 clk= ~clk;

initial

begin

data_in=0;

wr=0;

reset=0;

write_enable=0;

rd1=0;

rd2=0;

clk=0;

reset=1;

#10

reset=0;

@(posedge clk);

write_enable=1;

wr= 5'd1;

data_in= 32'd100;

@(posedge clk);

wr= 5'd2;

data_in= 32'd50;

@(posedge clk);
```

```
wr= 5'd3;

data_in= 32'd7;

@(posedge clk);

wr= 5'd4;

data_in= 32'd15;

@(posedge clk);

write_enable=0;

rd1= 5'd1;

rd2= 5'd2;

@(posedge clk);

rd1= 5'd2;

rd2= 5'd1;

@(posedge clk);

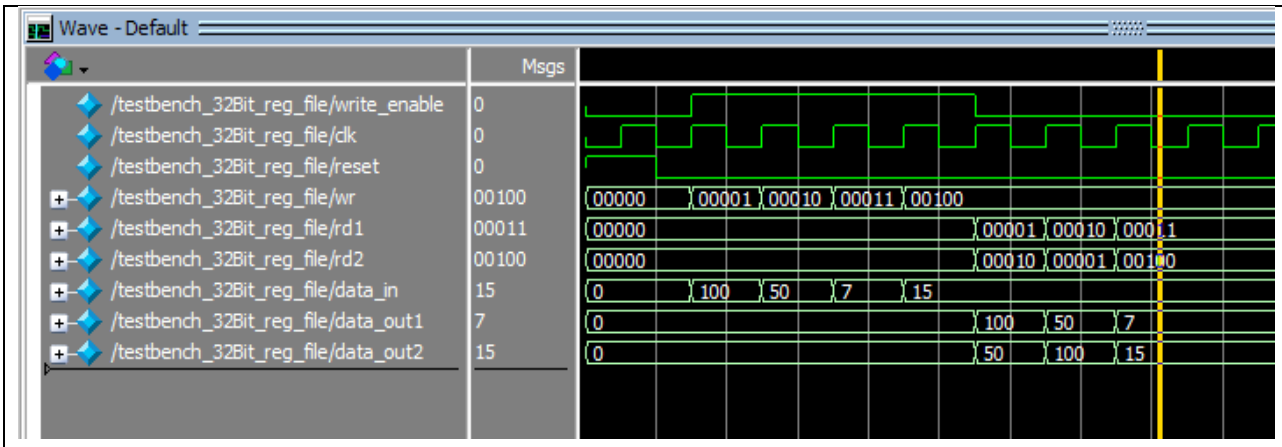
rd1= 5'd3;

rd2= 5'd4;

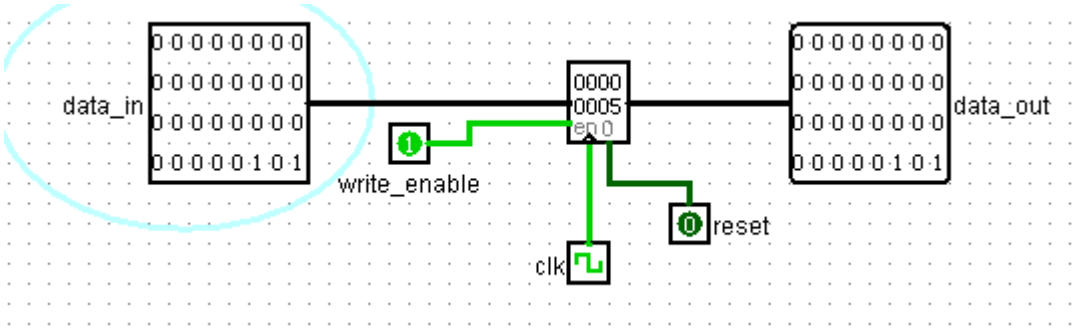
end

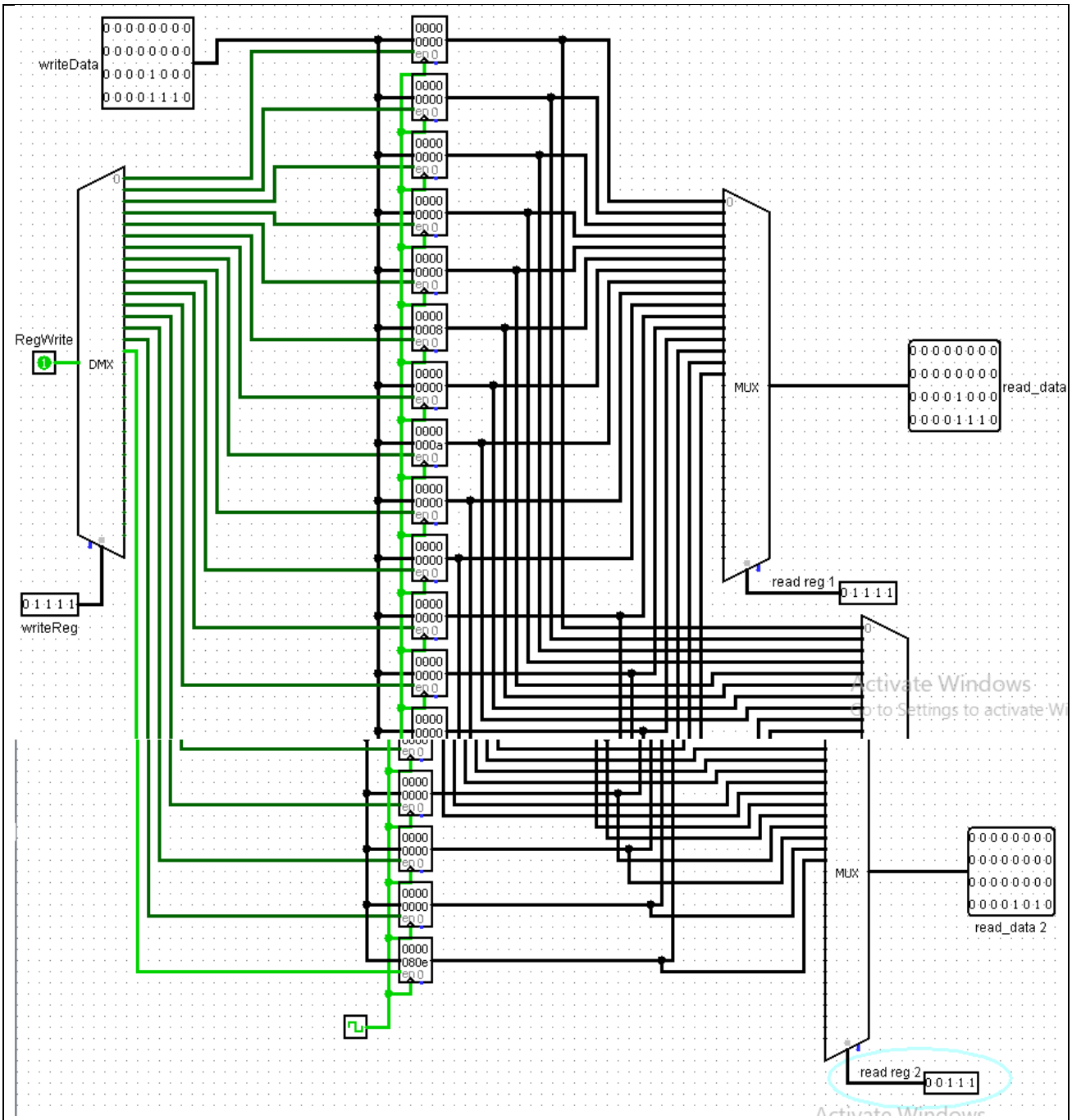
endmodule
```

Output:

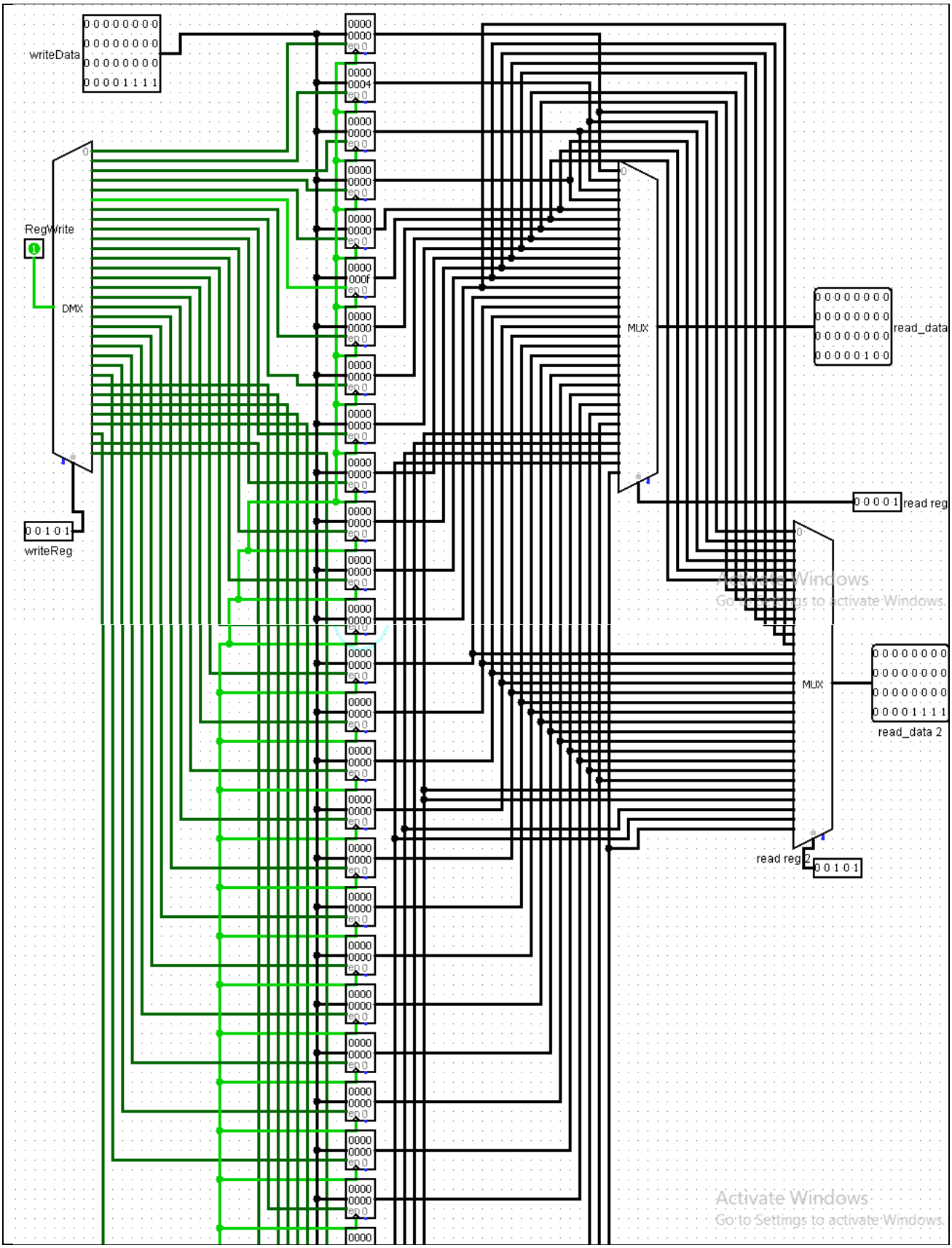


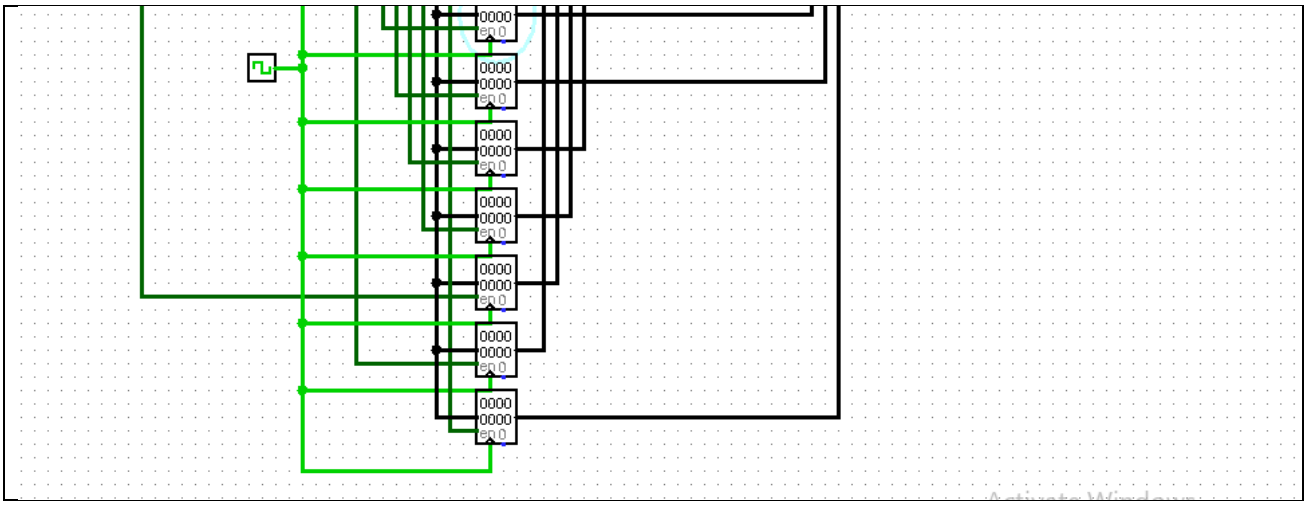
Task 4:

Description:
In this task, I have done the simulation of register using Logisim. Firstly, I have made 32 bits one register then I made 16 registers with 32 bits and in last I made 32 registers of 32 bits. Meanwhile, the data input relates to data probe, and enable is fed into decoder output, and clock is given to all the registers and then register outputs fed to MUX1 and MUX2 simultaneously. MUX selection lines were given as 5 bits as read reg 1 and 2 and decoder select line is write reg and given input as MUX is given outputs.
Output:
32-Bit Register:  **16 Registers of 32 Bits**



32 Bits of 32 Registers (32X32 Registers)


writeData
RegWrite
DMX
writeReg
MUX
read_data
read reg
MUX
read_data 2
read reg 2
Activate Windows
Go to Settings to activate Windows.


0000
0000
en 0